

STAR ESP32 Firmware Implementation Primer

ESP32-S3 Motor Control and Sensor Integration Guide

STAR Development Team

December 2025

Contents

1	Project Overview	3
1.1	Purpose	3
1.2	Current Implementation Status	3
2	Architecture Overview	3
2.1	Dependency Inversion Principle (DIP)	3
2.2	Directory Structure	3
2.3	Coding Style and Conventions	4
3	Motor Control Architecture	4
3.1	Motor Control Refactoring Plan	4
3.1.1	lib/star_motor (Proposed)	4
3.1.2	lib/star_encoder (Proposed)	5
3.1.3	lib/star_pid (Proposed)	5
3.2	Core Communication Interfaces	6
3.2.1	SPI Peripheral Driver	6
4	Extending the Bus Abstraction Layer	7
4.1	Task 1: Re-create GPIO Bus Driver	7
4.1.1	Modify Type Definition Files	7
4.1.2	Create GPIO Driver	8
4.1.3	Update Bus Config Factory	8
4.1.4	Update Build System	8
4.2	Task 2: Add ADC Bus Protocol	8
4.2.1	Modify Type Definition Files	8
4.2.2	Create ADC Driver	9
4.2.3	Update Bus Config Factory	9
4.2.4	Update Build System	9
4.3	Task 3: Implement Unified Bus Interface	9
4.3.1	Add Generic Functions to Bus Manager	9
4.4	Task 4: Final Bus Library Cleanup	10

5	Testing Plan	10
5.1	Unit Tests	10
5.2	Integration Tests	11
5.3	Acceptance Criteria	11
6	Build and Flash Instructions	11
6.1	Prerequisites	11
6.2	Build Commands	11
6.3	Configuration	11
7	References	12

1 Project Overview

1.1 Purpose

The `star-esp32-firmware` project provides real-time motor control, sensor integration, and communication with the Raspberry Pi 5 gateway for the STAR robot platform. It runs on the ESP32-S3 microcontroller using FreeRTOS.

1.2 Current Implementation Status

Component	Status	Notes
STAR Framework Core	Complete	DIP architecture, interfaces
Bus Abstraction Layer	Complete	I2C, SPI, 1-Wire, SMBus, GPIO
Essential Drivers	TODO	BQ7850 (BMS), DS18B20 (Temp) drivers required
Error Handling	Complete	Retry logic with exponential backoff
Pin Validation	Complete	GPIO conflict detection
Motor Control (MCPWM)	TODO	Requires refactoring per guide
SPI Peripheral (Pi5 comm)	Complete	Low-level communication layer is ready
PID Controller	TODO	To be created as <code>lib/star_pid</code>
Encoder Reading	TODO	To be created as <code>lib/star_encoder</code>

Table 1: Implementation Status Summary

2 Architecture Overview

2.1 Dependency Inversion Principle (DIP)

The firmware follows SOLID principles with emphasis on Dependency Inversion:

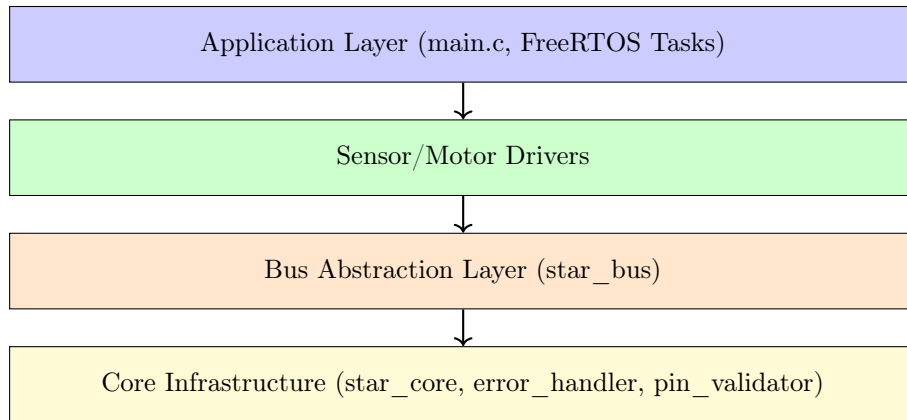


Figure 1: STAR Firmware Layered Architecture

2.2 Directory Structure

```

1 star-esp32-firmware/
2 |-- src/                                # Application entry point (currently empty)

```

```

3 |
4 |-- lib/
5 |   |-- star_core/           # Abstract interfaces
6 |   |-- star_error_handler/  # Error handling
7 |   |-- star_pin_validator/  # GPIO conflict detection
8 |   |-- star_bus/            # Unified bus abstraction
9 |   |-- star_motor/          # Motor, Encoder, and PID control
10 |   +-- star_bms_bq7850/     # Battery management
11 |
12 |-- platformio.ini          # Build configuration
13 +-- sdkconfig.*              # ESP-IDF configs

```

2.3 Coding Style and Conventions

To ensure long-term maintainability, consistency, and readability of the firmware, this project adheres to a strict set of coding conventions. All contributions must follow the rules outlined in the official project style guide.

The primary source of truth for all coding style is the `styleguide.txt` document located in the root of the `star-esp32-firmware` directory.

Adherence to this style guide is not optional; it is a critical requirement for all development within this project.

3 Motor Control Architecture

3.1 Motor Control Refactoring Plan

The `star_motor` library currently handles all motor-related functionality. For improved modularity, testability, and clarity, it is recommended to refactor this monolithic library into three distinct components as originally envisioned. The following sections outline the proposed architecture for each module.

3.1.1 `lib/star_motor` (Proposed)

This library should be responsible only for low-level motor control using the ESP32's MCPWM peripheral. Its sole focus is to abstract the hardware details of generating PWM signals.

1. Create a clean interface with motor handle and configuration structs.
2. Implement MCPWM initialization for 4 motors (2 MCPWM units, 2 operators each).
3. Configure center-aligned (up-down count) mode for reduced EMI.
4. Implement dead-time insertion to prevent H-bridge shoot-through.
5. Add fault input handling for overcurrent protection.

Key Parameters:

- PWM Frequency: 20 kHz (above audible range)
- Dead Time: 500ns (DRV8243 requirement)

- Duty Cycle Resolution: 0.1%

Listing 1: Proposed Motor Driver Interface

```

1 typedef struct {
2     mcpwm_unit_t unit;
3     mcpwm_timer_t timer;
4     gpio_num_t pwm_a;
5     gpio_num_t pwm_b;
6     gpio_num_t fault_pin;
7     float max_duty;
8 } star_motor_config_t;
9
10 esp_err_t star_motor_init(star_motor_handle_t* handle,
11                           const star_motor_config_t* config,
12                           star_error_interface_t* error_iface);
13 esp_err_t star_motor_set_duty(star_motor_handle_t* handle, float
14                               duty_cycle);
15 esp_err_t star_motor_brake(star_motor_handle_t* handle);
16 esp_err_t star_motor_coast(star_motor_handle_t* handle);

```

3.1.2 lib/star_encoder (Proposed)

This library would be dedicated to handling velocity feedback from the quadrature encoders.

1. Use the ESP32 PCNT (Pulse Counter) peripheral.
2. Configure for quadrature decoding (4x resolution).
3. Implement velocity calculation from the count delta over time.
4. Add overflow handling for the 16-bit hardware counter.

Encoder Specifications:

- Resolution: 341.2 counts/revolution
- Wheel diameter: 100mm
- Distance per count: 0.92mm

3.1.3 lib/star_pid (Proposed)

This library should be a generic, reusable PID controller for closed-loop control. It should contain no hardware-specific code.

Listing 2: Proposed PID Controller Interface

```

1 typedef struct {
2     float kp, ki, kd;           % Gains
3     float integral_limit;       % Anti-windup
4     float output_limit;         % Max output
5     float deadband;             % Ignore small errors
6 } star_pid_config_t;

```

```

7
8 void star_pid_init(star_pid_t* pid, const star_pid_config_t* config);
9 float star_pid_update(star_pid_t* pid, float setpoint, float measurement,
10                      float dt);
11 void star_pid_reset(star_pid_t* pid);

```

Initial Tuning Values (adjust experimentally):

- $K_p = 1.0$, $K_i = 0.5$, $K_d = 0.1$
- Loop rate: 1000 Hz
- Integral limit: ± 100

3.2 Core Communication Interfaces

3.2.1 SPI Peripheral Driver

The firmware implements SPI peripheral mode for communication with the Raspberry Pi 5.

- **Mode:** Peripheral on SPI2 (HSPI)
- **Protocol:** Custom binary packet structure
- **Performance:** DMA is used for efficient, non-blocking transfers

SPI Pin Mapping:

Signal	Pi5 GPIO	ESP32 GPIO
COPI	GPIO11	GPIO13
CIPO	GPIO9	GPIO12
SCLK	GPIO11	GPIO14
CS	GPIO8	GPIO15

Command Protocol:

Listing 3: SPI Command Protocol

```

1 // Command byte definitions
2 #define CMD_SET_VELOCITY      0x01 // Set wheel velocities
3 #define CMD_EMERGENCY_STOP   0x02 // Immediate stop
4 #define CMD_GET_ENCODERS     0x03 // Read encoder counts
5 #define CMD_GET_TELEMETRY    0x06 // Battery, temp, status
6
7 // Packet format: [CMD (1)] [LENGTH (1)] [DATA (N)] [CRC8 (1)]
8 typedef struct __attribute__((packed)) {
9     uint8_t command;
10    uint8_t length;
11    uint8_t data[32];
12    uint8_t crc8;
13 } spi_packet_t;
14
15 // Velocity command payload
16 typedef struct __attribute__((packed)) {

```

```

17     float left_velocity;    // m/s
18     float right_velocity;   // m/s
19     uint32_t timestamp;     // ms since boot
20 } velocity_cmd_t;

```

4 Extending the Bus Abstraction Layer

To fully realize the Dependency Inversion Principle and support all required hardware (including ADCs and GPIO-controlled multiplexers), the `star_bus` library must be extended. This section provides a detailed plan for this enhancement.

4.1 Task 1: Re-create GPIO Bus Driver

The GPIO bus driver is required to provide a standardized interface for controlling digital logic, such as the multiplexer for selecting ADC lines.

4.1.1 Modify Type Definition Files

1. **File:** `lib/star_bus/include/star_bus_common_types.h`
Add `k_star_bus_type_gpio` to the `star_bus_type_t` enum.
2. **File:** `lib/star_bus/src/star_bus_types.c`
Add the corresponding "GPIO" string to the `star_bus_type_strings` array.
3. **File:** `lib/star_bus/include/star_bus_protocol_types.h`
Add the necessary GPIO configuration and operations structs.

Listing 4: Additions to `star_bus_protocol_types.h`

```

1  /** @brief GPIO operations function pointers */
2  typedef struct {
3      esp_err_t (*write_digital)(const struct star_bus_config* config,
4                                uint8_t pin_index, uint8_t value);
5      esp_err_t (*read_digital)(const struct star_bus_config* config,
6                                uint8_t pin_index, uint8_t* value);
7      /* Other GPIO ops like interrupts can be added here */
8  } star_gpio_ops_t;
9
10 /** @brief GPIO bus configuration */
11 typedef struct {
12     gpio_num_t* pins;
13     uint8_t pin_count;
14     bool isr_installed;
15     star_gpio_ops_t ops;
16     /* Other config like callbacks can be added here */
17 } star_gpio_bus_config_t;
18
19 // Then, add star_gpio_bus_config_t to the star_bus_protocol_config_t union
20 typedef union {
21     // ... existing i2c, spi, etc. structs

```

```

20     star_gpio_bus_config_t gpio;
21 } star_bus_protocol_config_t;

```

4.1.2 Create GPIO Driver

1. Create `lib/star_bus/include/star_bus_gpio.h` to declare the public API for the GPIO bus, including the default ops initializer.
2. Create `lib/star_bus/src/star_bus_gpio.c`. This file will implement the GPIO logic using the ESP-IDF `driver/gpio.h` functions.

4.1.3 Update Bus Config Factory

1. **File:** `lib/star_bus/include/star_bus_config.h`
Add the function declaration for `star_bus_config_create_gpio()`.
2. **File:** `lib/star_bus/src/star_bus_config.c`
Implement `star_bus_config_create_gpio()` and add the `k_star_bus_type_gpio` case to the switch statements in `star_bus_config_init` and `star_bus_config_deinit`.

4.1.4 Update Build System

1. **File:** `lib/star_bus/CMakeLists.txt`
Add `"src/star_bus_gpio.c"` to the SRCS list.

4.2 Task 2: Add ADC Bus Protocol

This involves creating a new bus type for handling analog-to-digital conversions, following the established pattern in the library.

4.2.1 Modify Type Definition Files

1. **File:** `lib/star_bus/include/star_bus_common_types.h`
Add `k_star_bus_type_adc` to the `star_bus_type_t` enum before the count.
2. **File:** `lib/star_bus/src/star_bus_types.c`
Add the corresponding "ADC" string to the `star_bus_type_strings` array.
3. **File:** `lib/star_bus/include/star_bus_protocol_types.h`
Add the necessary ADC configuration and operations structs.

Listing 5: Additions to `star_bus_protocol_types.h`

```

1  /** @brief ADC operations function pointers */
2  typedef struct {
3      esp_err_t (*read_raw)(const struct star_bus_config* config, int*
4                          out_raw);
5      esp_err_t (*read_voltage)(const struct star_bus_config* config, int*
6                          out_voltage_mv);
7  } star_adc_ops_t;
8
9  /** @brief ADC bus configuration */

```



```

8  typedef struct {
9      adc_oneshot_unit_handle_t unit_handle;
10     adc_cali_handle_t cali_handle;
11     adc_channel_t channel;
12     adc_bitwidth_t bitwidth;
13     adc_atten_t atten;
14     star_adc_ops_t ops;
15 } star_adc_bus_config_t;
16
17 // Then, add star_adc_bus_config_t to the star_bus_protocol_config_t union
18 typedef union {
19     // ... existing i2c, spi, gpio, etc. structs
20     star_adc_bus_config_t adc;
21 } star_bus_protocol_config_t;

```

4.2.2 Create ADC Driver

1. Create `lib/star_bus/include/star_bus_adc.h` to declare the default ops initializer.
2. Create `lib/star_bus/src/star_bus_adc.c`. This file will implement the ADC logic using the ESP-IDF `adc_oneshot` and `adc_cali` drivers. It will define the functions for the `star_adc_ops_t` struct.

4.2.3 Update Bus Config Factory

1. **File:** `lib/star_bus/include/star_bus_config.h`
Add the function declaration for `star_bus_config_create_adc()`.
2. **File:** `lib/star_bus/src/star_bus_config.c`
Implement `star_bus_config_create_adc()` and add the `k_star_bus_type_adc` case to the switch statements in `star_bus_config_init` and `star_bus_config_deinit`.

4.2.4 Update Build System

1. **File:** `lib/star_bus/CMakeLists.txt`
Add `"src/star_bus_adc.c"` to the SRCS list.

4.3 Task 3: Implement Unified Bus Interface

Create a true abstraction layer in the bus manager so that application code does not need to call protocol-specific functions.

4.3.1 Add Generic Functions to Bus Manager

1. **File:** `lib/star_bus/include/star_bus_manager.h`
Declare new high-level, generic functions.
2. **File:** `lib/star_bus/src/star_bus_manager.c`
Implement these functions. Each function will find the bus by name, verify its type, and call the appropriate function pointer from the bus's `ops` struct.

Listing 6: Proposed Generic Interface in star_bus_manager.h

```

1  /**
2   * @brief Write a digital value to a bus (e.g., GPIO).
3   */
4  esp_err_t star_bus_write_digital(star_bus_manager_t* manager, const char*
    bus_name, uint8_t value);
5
6  /**
7   * @brief Read an analog value from a bus (e.g., ADC).
8   * @param[out] out_voltage_mv The resulting voltage in millivolts.
9   */
10 esp_err_t star_bus_read_analog_mv(star_bus_manager_t* manager, const char*
    bus_name, int* out_voltage_mv);
11
12 /**
13  * @brief Perform a standard I2C/SPI write operation.
14  */
15 esp_err_t star_bus_write(star_bus_manager_t* manager, const char* bus_name
    , uint8_t reg_addr, const uint8_t* data, size_t len);
16
17 /**
18  * @brief Perform a standard I2C/SPI read operation.
19  */
20 esp_err_t star_bus_read(star_bus_manager_t* manager, const char* bus_name,
    uint8_t reg_addr, uint8_t* data, size_t len);

```

This approach provides a clean, abstract, and extensible bus management system that fully adheres to the Dependency Inversion Principle.

4.4 Task 4: Final Bus Library Cleanup

As a final cleanup step, the unnecessary 1-Wire peripheral driver should be removed.

1. Delete lib/star_bus/src/star_bus_onewire_peripheral.c.
2. Delete the corresponding header file if it exists in lib/star_bus/include/.
3. Remove any references from the CMakeLists.txt file.

Reasoning: The ESP32 will only act as a controller on the 1-Wire bus to communicate with the DS18B20 sensor. It will never need to act as a 1-Wire peripheral device.

5 Testing Plan

5.1 Unit Tests

1. **PID Controller:** Test with simulated plant, verify convergence
2. **Encoder Driver:** Test count accuracy, overflow handling
3. **Motor Driver:** Test PWM generation, dead-time
4. **SPI Protocol:** Test packet parsing, CRC validation

5.2 Integration Tests

1. **Motor + Encoder:** Closed-loop step response test
2. **SPI + Motor:** Command-to-motion latency measurement
3. **Full System:** Velocity tracking accuracy at various speeds

5.3 Acceptance Criteria

Metric	Target
PID loop rate	1000 Hz $\pm$ 1%
Velocity tracking error	< 5% at steady state
Command latency (SPI)	< 1ms
Emergency stop time	< 10ms

6 Build and Flash Instructions

6.1 Prerequisites

```

1 # Install PlatformIO
2 pip install platformio
3
4 # Install ESP-IDF (handled by PlatformIO)
5 pio pkg install

```

6.2 Build Commands

```

1 # Build for ESP32-S3
2 pio run -e esp32s3
3
4 # Build for ESP32-WROOM (dev board)
5 pio run -e esp32_wroom
6
7 # Upload firmware
8 pio run -e esp32s3 --target upload
9
10 # Monitor serial output
11 pio device monitor -b 115200

```

6.3 Configuration

Edit `sdkconfig.esp32s3` for production settings:

- Enable brownout detection
- Set CPU frequency to 240 MHz
- Configure flash size (16 MB)
- Enable PSRAM if needed

7 References

- ESP-IDF MCPWM Documentation: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/api-reference/peripherals/mcpwm.html>
- ESP-IDF SPI Peripheral: https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/api-reference/peripherals/spi_slave.html
- FreeRTOS Task Notifications: <https://www.freertos.org/RTOS-task-notifications.html>
- DRV8243 Datasheet: TI SLVSN4
- STAR FTP Section 16: Embedded Implementation
- STAR Software Architecture Primer: Section 12 (ESP32 Firmware)